GREEN INSIGHTS

ML pipeline: what changed, how it works, what it outputs, and how the team runs it

A practical engineering walkthrough for teammates, reviewers, and demo presenters

Repository	github.com/dev-rjt/FineShyt
Branch	feature/ml-json-contract-v1
Commit	9135421
Contract	Opportunity JSON schema v1.0.0
Prepared	10 October 2026

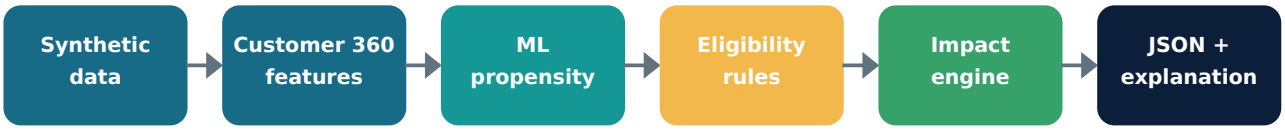
1. Executive summary

The original archive had a useful synthetic-data and modelling core, but its product codes and flat CSV-style output did not match the team's final integration contract. Ineligible customers also received no propensity, which made it impossible to demonstrate the required 'high score but blocked by rules' safety case.

The core design

Every customer gets four product evaluations. ML estimates take-up propensity. Product rules decide whether an opportunity is allowed. Deterministic formulas estimate impact. A contract builder packages the result and validates it before writing output.

One traceable path from data to an approved customer action



ML predicts | rules decide | deterministic code calculates | explanation uses approved facts

What was changed

Area	Before	Now
Product codes	GREEN_HOME, PAPERLESS	GREEN_HOME_LOAN, PAPERLESS_BANKING
Coverage	Predictions only for eligible candidates	Propensity for every customer-product pair
Model choice	Always LightGBM	Validation-selected LightGBM / logistic-regression blend
Output	Flat recommendation CSV	Nested JSON v1.0.0 validated at runtime
Safety	Boolean exclusions	ELIGIBLE, INELIGIBLE, or MANUAL_REVIEW with decision codes
Explanations	Not implemented	Grounded deterministic fallback using approved structured facts
Testing	Evaluation only	Safety tests plus schema validation for every record

2. End-to-end architecture

The pipeline follows the problem statement's separation of responsibility. Components do not overwrite one another's decisions, and the final record retains component versions for audit.

Stage	Reads	Produces	Key file
1. Data generation	Seeded configuration	Customers, transactions, products, vehicles, energy, digital use, labels	data_generation.py
2. Transaction model	Raw merchant descriptions	One of eight mutually exclusive categories	txn_classifier.py
3. Customer 360	All source tables + predicted categories	One feature row per customer	features.py
4. Propensity models	Customer features + future take-up labels	Propensity, confidence, local reason codes	propensity.py
5. Eligibility	Customer facts and product holdings	Tri-state rule decision with evidence	rules.py
6. Impact	Approved activity inputs + factor table	Deterministic impact or insufficient-data result	carbon_engine.py
7. Contract	All structured sections	Validated JSON v1.0.0 + explanation + audit	contract.py
8. Orchestration	Configuration and CLI arguments	Models, metrics, JSONL, flat summary	run_pipeline.py

Why this matters The design prevents an ML score or explanation model from silently becoming a financial eligibility decision. Each layer can be tested, replaced, and audited independently.

3. Synthetic data and label generation

No real or confidential banking data is used. The default run creates 5,000 customers and joins all records through customer_id. A fixed seed makes a fresh process reproducible.

Dataset	Examples	How it contributes
Customers	Age, income band, postcode, home ownership, digital use, sustainability preference	Segments and demographic / behavioural features
Transactions	Date, raw merchant text, amount, debit or credit	Spend patterns and transaction-category model
Products	Mortgage, vehicle loan, savings, current green products	Signals and rule exclusions
Vehicles	Fuel type, vehicle age, mileage	EV propensity, eligibility, transport impact
Energy	Monthly electricity and gas kWh	Green Home and Solar features / impact
Digital usage	Paper statements, digital payments, branch visits	Paperless propensity and evidence
Reference data	Product catalogue and emissions factors	Stable product mapping and deterministic formulas

How labels are created

For each required product, take-up is sampled from a documented latent logistic process. Observable signals receive plausible weights and random noise represents unobserved customer factors. Labels occur only after the 2025-12-31 feature cutoff, within a six-month outcome window.

Product	Examples of positive synthetic signals
EV Finance	Mileage, fuel spend, sustainability preference, vehicle loan, older vehicle
Green Home Loan	Heating use, sustainability preference, home-improvement activity, mortgage
Solar Finance	Electricity use, sustainability preference, income, rural location
Paperless Banking	Paper share, digital transactions, low branch use, digital engagement

Leakage control Customers, not individual rows, are split 70% train, 15% validation, and 15% test. A customer's transactions and labels never cross split boundaries.

4. Transaction categorisation

Every transaction must belong to exactly one sustainability category: Fuel, Electricity, Gas, Public Transport, Home Improvement, Automotive, Utilities, or Other.

Component	Implementation
Text preparation	Uppercase, remove transaction-feed prefixes and numbers, collapse whitespace
Model	Character TF-IDF n-grams (2-5) plus class-balanced logistic regression
Training unit	Unique cleaned descriptions with frequency-based sample weights
Rule baseline	Ordered keyword rules with documented boundaries
Held-out evaluation	Customer-level validation and test customers only
Downstream use	Predicted categories, never ground-truth categories, feed Customer 360

Important category boundaries

- EV charging transactions are treated as Fuel; forecourt shop purchases are Other.
- Supermarket petrol stations are Fuel, while normal supermarket purchases are Other.
- MOT, tyres, tax, parts, and car wash are Automotive, not Fuel.
- Electricity and Gas are separated from general Utilities such as water and broadband.

0.999	0.999	0.837	0.811
MODEL MACRO-F1	MODEL ACCURACY	RULE BASELINE MACRO-F1	RULE BASELINE ACCURACY

These figures come from the latest checked local smoke run and are intentionally reported alongside the simpler rule baseline.

5. Product propensity models

A separate binary model is trained for each required product. The score answers: 'How likely is this customer to take up this product within the defined outcome window?' It does not answer whether the customer is financially eligible.

Step	Method
Candidate training population	Only customers who satisfy the synthetic candidate definition
Complex model	Regularised LightGBM ensemble with five seeded members
Simple baseline	Median imputation, standardisation, logistic regression
Hyperparameters	Five-fold training-only CV using log loss and early stopping
Probability calibration	Platt calibration of tree scores on validation data
Champion selection	Choose a validation-log-loss blend from 0%, 25%, 50%, 75%, or 100% LightGBM
Operating threshold	Validation threshold maximizing F1; eligibility still runs separately
Untouched assessment	AUC, PR-AUC, log loss, Brier, lift at top 10%, and bootstrap AUC interval on test

Why the blend is better than 'always use the fanciest model'

Small synthetic candidate groups can favour a linear model. The validation-selected blend allows logistic regression to dominate when it predicts held-out probabilities better, while still retaining tree capacity where nonlinear interactions help.

Interpretation Propensity is a calibrated behavioural estimate from 0 to 1. It is not credit approval, affordability, suitability, financial advice, or eligibility.

6. Confidence and explainability

Confidence is deliberately separate from propensity. A customer can have high propensity but low confidence when important inputs are absent, unusual, or unstable across models.

Confidence component	Weight	Meaning
Model agreement	40%	Five LightGBM members and logistic regression should give similar estimates
Key-feature completeness	30%	Product-critical inputs such as mileage or paper share are present
In-distribution support	30%	Key values fall within the 1st-99th percentile training range

Reason codes

Local positive contributions from the tree and linear models are normalised, blended using the champion weight, and mapped to fixed reason codes. Each reason includes a human label, local importance, source feature, value, unit, and comparison.

Reason code	Evidence example
HIGH_FUEL_SPEND	fuel_spend_m, GBP/month, above population median
HIGH_ANNUAL_MILEAGE	annual_mileage, miles/year, above population median
HIGH_ELECTRICITY_USE	elec_kwh_yr, kWh/year, above population median
PAPER_STATEMENTS_ACTIVE	paper_share, ratio, above configured threshold
HIGH_DIGITAL_ACTIVITY	digital_tx_m, transactions/month, above population median

Audit benefit A recommendation can be traced from a reason code to its actual feature value instead of showing only a generic feature-importance chart.

7. Eligibility and recommendation safety



A high ML score can never override a failed rule.

Product	Illustrative blocking conditions
EV Finance	No vehicle; already owns an EV without a replacement case; outside age criteria; already holds GP01
Green Home Loan	No home ownership or mortgage; already holds GP02
Solar Finance	Not a homeowner; already holds GP03
Paperless Banking	Already mostly digital; missing statement data; already holds GP04

Tri-state result

- ELIGIBLE: rules pass. The threshold may then produce RECOMMEND or DO_NOT_RECOMMEND.
- INELIGIBLE: at least one rule fails. The customer is never shown an offer.
- MANUAL_REVIEW: required information is unknown. The record requests missing data instead of inventing it.

Proven safety case The validated output contains an EV case with propensity 0.9408 that is still INELIGIBLE, DO_NOT_RECOMMEND, and customer_visible=false. This proves that rules override the score.

8. Deterministic sustainability impact

Impact is calculated by ordinary code, never by the ML model or an LLM. If a required input is missing, the engine returns INSUFFICIENT_DATA and null values rather than zero or an invented estimate.

Product	Primary method	Reported result
EV Finance	Current vehicle km x vehicle factor minus EV km x energy intensity x grid factor	Potential kgCO2e avoided/year
Green Home Loan	Metered energy baseline versus assumed gas/electricity retrofit reduction	Potential kgCO2e avoided/year + energy saved
Solar Finance	PV generation capped at annual electricity use, multiplied by grid factor	Potential kgCO2e avoided/year + kWh generated
Paperless Banking	Paper statements and sheets moved to digital	Statements/year and sheets/year; digital adoption separate

Every available impact contains

- A lower, central, and upper estimate instead of false precision.
- Unit, one-year time horizon, method ID, formula description, and calculation inputs.
- Explicit assumptions and the relevant emissions-factor provenance.
- A calculation timestamp and status.

Production warning The current factor table is a documented prototype. Replace it with institution-approved, current, versioned emissions factors before real use. Eligibility rules are also illustrative, not authorised lending policy.

9. Final JSON contract

One line in the JSONL output represents one customer plus one product opportunity. With the default 5,000 customers and four products, a full run writes 20,000 records. Every record is checked against opportunity_v1.schema.json before it is saved.

Section	Owner and purpose
schema_version + request metadata	Integration layer: version, request ID, timestamps, cutoff
customer	Data pipeline: customer ID, segment, region, completeness
opportunity	Product catalogue: canonical ID, code, name, category
prediction	ML: propensity, confidence, model version, reason evidence
eligibility	Rules: status, rule version, pass/fail/unknown decisions
impact	Impact engine: status, range, method, assumptions, factors, inputs
recommendation	Decision engine: recommend / block / review, priority, action, visibility
explanation	Verified customer and manager wording from approved fields
audit	Component versions, warnings, errors, review flag, data split

Compact shape example

```
{
  "customer": {"customer_id": "C13032", "region": "..."},
  "opportunity": {"product_code": "EV_FINANCE"},
  "prediction": {"propensity": 0.9408, "confidence": 0.88, "reason_codes": [...]},
  "eligibility": {"status": "INELIGIBLE", "failed_rules": [...]},
  "impact": {"status": "NOT_APPLICABLE"},
  "recommendation": {"decision": "DO_NOT_RECOMMEND", "customer_visible": false},
  "explanation": {...}, "audit": {...}
}
```

10. What the pipeline outputs

1000	4000	4	1.0.0
CUSTOMERS IN LATEST LOCAL ARTIFACT	CUSTOMER-PRODUCT RECORDS	REQUIRED PRODUCTS	JSON CONTRACT VERSION

Path	Purpose	Committed?
ml/outputs/opportunities_all.jsonl	All canonical customer-product records	No - regenerated locally
ml/outputs/test_predictions.jsonl	Canonical records for untouched test customers	No - regenerated locally
ml/outputs/recommendations_summary.csv	Flat dashboard-friendly decision summary	No - regenerated locally
ml/outputs/evaluation_report.json	Model metrics, baselines, blend, threshold, intervals	Yes
ml/models/propensity_models.joblib	Four trained propensity model objects	Yes
ml/models/txn_classifier.joblib	TF-IDF vectorizer and transaction classifier	Yes
ml/data/customer_360.csv	Generated feature table	No - regenerated locally
ml/contracts/opportunity_v1.schema.json	Runtime JSON validator contract	Yes

Latest local recommendation counts

Product	RECOMMEND records
EV_FINANCE	69
GREEN_HOME_LOAN	45
SOLAR_FINANCE	111
PAPERLESS_BANKING	123

Generated JSONL and large CSV files are deliberately ignored by Git. This keeps the branch small while preserving one-command reproducibility.

11. Evaluation and how to read it

The table below reflects the latest checked local smoke-run report. Small test groups make these values uncertain; they are pipeline evidence, not claims of production performance. A full default run creates larger groups.

Product	Test AUC	Test PR-AUC	Log loss	Lift@10%	LGB / LR	Test n
EV_FINANCE	0.856	0.600	0.341	2.91x	1.00 / 0.00	64
GREEN_HOME_LOAN	0.859	0.386	0.262	5.38x	0.00 / 1.00	43
SOLAR_FINANCE	0.577	0.206	0.589	1.79x	1.00 / 0.00	43
PAPERLESS_BANKING	0.796	0.622	0.417	4.12x	0.25 / 0.75	77

Metric guide

Metric	Interpretation
ROC-AUC	Ranking quality across positive and negative examples; 0.5 is random
PR-AUC	More informative when take-up is rare; higher is better
Log loss	Quality of predicted probabilities; strongly penalises confident mistakes; lower is better
Brier score	Mean squared probability error; lower is better
Lift@10%	Positive rate among the top 10% divided by overall positive rate
Bootstrap interval	Uncertainty range for test AUC from repeated resampling

Honest interpretation

Synthetic metrics show that the code can learn the documented relationships and reproduce a rigorous evaluation workflow. They do not prove performance on real Lloyds customers.

12. How teammates run it

Clone or fetch the repository, switch to the shared feature branch, create an isolated Python environment, install dependencies, and run either the smoke test or the full pipeline.

Get the branch

```
git fetch origin
git switch feature/ml-json-contract-v1
```

Install once

```
python3 -m venv .venv
.venv/bin/pip install -r ml/requirements.txt
```

Quick verification

```
.venv/bin/python ml/run_pipeline.py --customers 1000
PYTHONPATH=ml .venv/bin/python -m unittest discover -s ml/tests -v
```

Full default run

```
.venv/bin/python ml/run_pipeline.py
```

Where to look after the run

- Start with ml/outputs/evaluation_report.json for model quality and baseline comparisons.
- Open ml/outputs/recommendations_summary.csv for dashboard integration.
- Read one line from ml/outputs/opportunities_all.jsonl for the complete API object.
- Use ml/contracts/opportunity_v1.schema.json in every service boundary.

Runtime note	The full run processes a large synthetic transaction table and is hardware-dependent. Use --customers 1000 for daily development and the default 5,000 for the final reproducibility check.

13. Limitations, ownership, and next steps

Current limitation	Required production action
Synthetic customers and synthetic take-up	Train and validate on approved, de-identified or lawfully usable data; document drift and representativeness
Prototype emissions factors	Replace with current institution-approved, source-year and version-controlled factors
Illustrative eligibility rules	Replace with authorised product, affordability, suitability, and compliance processes
Deterministic fallback explanation	Optionally add an LLM only for wording, plus claim verification and fixed fallback
Small candidate groups in smoke runs	Use larger samples, repeated validation, subgroup checks, and temporal validation
Synthetic text patterns are easy	Test merchant classification on realistic unknown merchants, ambiguous descriptions, and distribution shift
No serving layer in this module	Expose the contract through FastAPI and connect customer, manager, and audit views

Suggested team ownership

Owner	Primary responsibility
Data	Schema, generation, sources, cutoff, outcome window, feature store
ML	Transaction model, propensity models, calibration, confidence, metrics
Rules / impact	Eligibility versions, factor catalogue, formulas, unit tests
LLM / responsible AI	Prompt, verifier, fallback, groundedness evaluation, limitations
Integration / UX	API, dashboard, audit drill-down, monitoring, demo

Bottom line

The branch now provides a complete, reproducible and auditable ML core: it predicts clearly, blocks unsafe recommendations, calculates impact transparently, and emits one stable JSON contract for the rest of the team.

Shared branch: https://github.com/dev-rjt/FineShyt/tree/feature/ml-json-contract-v1
Pull request: https://github.com/dev-rjt/FineShyt/pull/new/feature/ml-json-contract-v1